

Chart Digitizer

Files, Scripts & Code Explainer

University of Naples Federico II
MSc Data Science
Supervisor: Prof. Schiano Lo Moriello

June 2026

1. Project File Map

The entire project lives inside the `chart_digitizer/` directory. Below is the complete file tree with a one-line description of every file.

<code>chart_digitizer/</code>	Project root
<code>■■■ app.py</code>	Flask web server — REST API backend
<code>■■■ start.sh</code>	One-command local launch script
<code>■■■ Dockerfile</code>	Docker container definition for deployment
<code>■■■ requirements.txt</code>	Python dependency list
<code>■■■ generate_explainer.py</code>	Generates the technical explainer PDF
<code>■■■ generate_code_explainer.py</code>	Generates this document
<code>■■■ pipeline/</code>	
<code>■ ■■■ pipeline.py</code>	Core CV extraction pipeline
<code>■■■ synthetic/</code>	
<code>■ ■■■ synthetic_pdf_generator.py</code>	Generates synthetic test PDFs with ground truth
<code>■ ■■■ synthetic_generator.py</code>	Legacy PNG-based chart generator
<code>■■■ tests/</code>	
<code>■ ■■■ evaluate_accuracy.py</code>	Benchmarks pipeline output vs ground truth
<code>■■■ frontend/</code>	
<code>■■■ index.html</code>	Drag-and-drop single-file web UI

2. synthetic_pdf_generator.py

`synthetic/synthetic_pdf_generator.py`

Role: Generates realistic AMS milking PDFs for pipeline testing, with embedded text and known ground truth.

Why this file exists

Only one real PDF was provided by the company. To test and measure pipeline accuracy rigorously, we need PDFs where the exact curve coordinates are known in advance. This script generates an unlimited number of such PDFs.

Key design decision — Matplotlib PDF backend

Matplotlib is normally used to render charts to screen or PNG. By switching its backend to pdf, it writes a real PDF with vector graphics and **embedded searchable text**. This means PyMuPDF can later extract the animal ID, date, and time directly — exactly as it does from the real company PDF.

```
import matplotlib
matplotlib.use('pdf') # switch to PDF backend before importing pyplot
import matplotlib.pyplot as plt
```

Curve generators

generate_flusso() — milk flow rate (blue curve):

Creates a bell-shaped curve: rises quickly to a peak, holds briefly, then falls to zero. Peak height, timing, and fall width are randomised. A taper function forces the curve to zero at the start and after the session duration. Gaussian noise is

added for realism.

generate_pressione() — vacuum pressure (green curve):

Creates a plateau curve: rises steeply from zero, holds at a high value (~7–9), then drops sharply at the end of the session. Ripple noise added throughout.

Both functions support a `flat=True` mode for edge-case testing (dry cows or unit-off scenarios).

render_chart()

Draws one chart onto a Matplotlib Axes object. The critical detail is how the header text is placed. Two `ax.annotate()` calls are used with `xycoords='axes fraction'` — this anchors text relative to the axes box, so it always appears just above the top spine regardless of which grid row the chart occupies.

```
ax.annotate('Numero Azienda 07010142 Animale: 1001 ...',
            xy=(0, 1), xycoords='axes fraction',
            xytext=(0, 12), textcoords='offset points',
            fontsize=5, family='monospace')
```

Three scenarios

Scenario	Pages	Charts	What it tests
baseline	1	8	Normal sessions — happy path
multipage	3	24	Multi-page iteration + CSV filename collision handling
edge	2	16	Flat curves, short sessions, duplicate animal IDs

Ground truth output: a `ground_truth.json` file is saved alongside the PDFs. It contains the exact (time, value) arrays for every curve on every chart — used by the accuracy evaluator.

3. synthetic_generator.py

[synthetic/synthetic_generator.py](#)

Role: Legacy PNG-based chart generator. Kept for visual inspection and backwards compatibility.

This was the first generator built. It uses the same curve generation logic but renders to PNG instead of PDF. Because PNGs have no embedded text, the pipeline must use Tesseract OCR to read metadata — which is slower and less reliable than native text extraction. The PDF generator supersedes this for all testing purposes, but the PNG generator remains useful for quickly previewing what synthetic charts look like.

Usage: `python synthetic_generator.py --pages 2 --out ./output --seed 42`

4. pipeline/pipeline.py

[pipeline/pipeline.py](#)

Role: The core of the project. Extracts Flusso and Pressione curve data from every chart in a PDF or image.

This file contains eight distinct functional components, each handling one step of the extraction pipeline. They are called in sequence by the top-level `run_pipeline()` function.

load_pages() *Input handler.*

Accepts a file path. If the file is a PDF, PyMuPDF rasterises each page to a BGR image at 150 DPI. If the file is a PNG or JPEG, OpenCV loads it directly. Returns a list of NumPy arrays — one per page.

extract_metadata_from_pdf_page()

Native PDF text extraction — no OCR.

Calls PyMuPDF's `get_text('blocks')` to retrieve all text blocks with their bounding box coordinates. Scans for blocks containing 'Animale:' and merges them with nearby blocks to capture 'Data:' and 'Ora:' (which sometimes appear in a separate block). Sorts results by Y-then-X position to match the grid order. Returns 8 metadata dicts.

slice_page_with_pdf_boundaries()

Accurate cell slicing using PDF text positions.

Instead of dividing the page into equal-height rows (which fails when chart heights vary), this function uses the actual Y coordinates of the embedded header text to find where each row starts. It then cuts the page into 8 cells — 4 rows x 2 columns. Falls back to equal-height slicing for PNG inputs.

detect_plot_area()

Axis frame detection.

Binaries the chart cell and uses morphological kernels to detect long horizontal and vertical lines. The innermost detected lines define the plot area bounding box: `x_left`, `x_right`, `y_top`, `y_bottom`. These pixel coordinates are passed to `AxisTransform`.

AxisTransform class

Pixel-to-data coordinate mapping.

A simple class initialised with the plot area pixel bounds and the known data ranges (X: 0-10 min, Y: 0-9 for both axes). The `px_to_data()` method applies linear interpolation. Y is inverted because image coordinates increase downward while data Y increases upward.

extract_curve_pixels()

HSV colour isolation.

Converts the plot region from BGR to HSV colour space, then applies `cv2.inRange()` with tuned HSV bounds for blue (Flusso) and green (Pressione). Morphological open-then-close cleans up noise. Returns pixel (x,y) coordinates of the curve.

pixels_to_curve()

Pixel cloud to ordered curve.

For each X column in the pixel cloud, takes the median Y value — robust to anti-aliasing where a curve spans 2-3 pixels vertically. Interpolates across columns where no pixel was detected. Samples 300 evenly-spaced points and maps each through `AxisTransform` to get (time_min, value) pairs. Curves are anchored to the plot origin (0, 0), and the flow curve is extended flat to the session end once it has returned to the axis — recovering segments drawn over the black axis lines that the colour mask cannot see.

process_chart()

Single-chart orchestrator.

Calls `detect_plot_area`, `AxisTransform`, `extract_curve_pixels`, and `pixels_to_curve` for one cell. Combines both curves into a Pandas DataFrame and writes the CSV. Uses metadata from the PDF text extraction (or OCR fallback). Handles filename collisions by appending a time suffix.

`run_pipeline()` *Top-level entry point.*

Iterates over pages. For PDF inputs: calls `extract_metadata_from_pdf_page` and `slice_page_with_pdf_boundaries`. For image inputs: calls `ocr_full_page` and `slice_page_into_charts`. Then calls `process_chart` for each of the 8 cells per page.

5. tests/evaluate_accuracy.py

[tests/evaluate_accuracy.py](#)

Role: Benchmarks the pipeline's output against synthetic ground truth. Produces a quantitative accuracy report.

How it works:

Loads `ground_truth.json` produced by the synthetic PDF generator. For each chart, finds the matching CSV by animal ID and date. Then for each curve:

1. Takes the ground truth (time, value) arrays from JSON.
2. Takes the pipeline's extracted (time, value) arrays from CSV.
3. Interpolates the ground truth at the pipeline's time points using `np.interp()`.
4. Computes $MAE = \text{mean}(|\text{predicted} - \text{ground_truth}|)$.
5. Computes $\text{coverage} = (t_{\text{max}} - t_{\text{min}}) / (gt_t_{\text{max}} - gt_t_{\text{min}})$.

Metrics reported:

- Detection rate — how many charts had both curves successfully extracted
- MAE mean, median, and max — for both Flusso and Pressione independently
- Coverage mean — what fraction of the X-axis was captured

Results on 48 synthetic charts (6 pages, 3 PDFs):

Metric	Flusso	Pressione
Detection rate	100%	100%
MAE — median	0.07 kg/min	0.12×0.1 bar
MAE — mean	0.44 kg/min	0.27×0.1 bar
Coverage — mean	98.8%	99.3%
Warnings	0	0

6. app.py

[app.py](#)

Role: Flask web server. Wraps the pipeline in a REST API so it can be driven from a browser.

Job model

Each upload creates a job with a unique UUID. Jobs are stored in a Python dict in memory. The pipeline runs in a **background thread** so the HTTP response returns immediately and the browser can poll for progress.

API endpoints

POST /upload	Save the uploaded PDF/PNG, create a job ID, start pipeline in background thread, return job ID.
GET /status/	Return job status (queued/running/done/error), list of output CSV filenames, and any warnings.
GET /download//	Stream a single CSV file to the browser as a download.
GET /download-all/	Zip all CSVs for a job and stream the ZIP archive.
GET /	Serve index.html (the frontend).

7. frontend/index.html

[frontend/index.html](#)

Role: The entire user interface in a single file. No framework, no build step, no dependencies.

UI flow — step by step:

1. User drops a PDF onto the drop zone or clicks to browse.
2. Clicking **Run Extraction** sends a multipart POST to /upload.
3. JavaScript receives the job ID and starts polling /status/<job_id> every 1.2 seconds.
4. A progress bar animates between 15% and 85% while the pipeline is running.
5. When status is done, the file list is rendered — one download link per CSV plus a Download all as ZIP button.
6. Any pipeline warnings (e.g. OCR fallback) appear in an amber box below.

The entire UI is ~250 lines of vanilla HTML/CSS/JS. No React, no Vue, no npm. This keeps the project dependency-free on the frontend and trivially portable.

8. start.sh & Dockerfile

[start.sh](#)

Role: One-command local launch script for development use.

Activates the Python virtual environment if present, then starts Flask on port 5001. Port 5001 is used instead of 5000 to avoid conflict with macOS AirPlay Receiver which occupies port 5000 by default.

```
#!/bin/bash
PORT=${1:-5001}
source ../venv/bin/activate
echo 'Open your browser at: http://localhost:$PORT'
PORT=$PORT python3 app.py
```

[Dockerfile](#)

Role: Defines the production container for self-hosted deployment on the farm's network.

Uses python:3.11-slim as base. Installs Tesseract OCR at the system level (required for PNG fallback), then installs all Python dependencies. Exposes port 5000 and runs python app.py on container start.

```
FROM python:3.11-slim
RUN apt-get update && apt-get install -y tesseract-ocr libgl1 libglu1-mesa-dev
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

9. How All Files Connect

The diagram below shows how data flows through the system from a user uploading a PDF to receiving CSV files in their browser.

User (browser)	→	frontend/index.html
frontend/index.html	→	POST /upload (app.py)
app.py	→	pipeline/pipeline.py (background thread)
pipeline.py	→	PyMuPDF (text extraction + rasterisation)
pipeline.py	→	OpenCV (cell slicing + colour isolation)
pipeline.py	→	NumPy (coordinate mapping)
pipeline.py	→	Pandas (CSV write)
app.py	→	GET /status → frontend shows file list
frontend	→	GET /download → CSV saved to user's machine